

Label-Efficient Learning with Unsupervised Methods

Clustering, Semi-Supervised Learning and Anomaly Detection,
served as an end-to-end MLOps system

Xaysomvang (Tom) Thammavong · Technical report, October 2026

Code: <https://github.com/ToMooGo/unsupervised-learning-mlops>

Labelled data is expensive; unlabelled data is cheap. This report develops three ways of using unsupervised learning to get more out of fewer labels, following the methods of G eron, *Hands-On Machine Learning* (2nd ed., Chapters 8 and 9), and derives the mathematics behind each step. Part A uses K-Means distances as features for a classifier; Part B labels only cluster-representative images and propagates their labels; Part C detects anomalous inputs with a Gaussian mixture density. The final section describes how the three models are trained, gated, deployed and monitored. The numbers quoted here are produced by the pipeline (`reports/metrics.json`) or by the executed notebooks in the repository.

Contents

1	Data and evaluation protocol	2
2	Part A: K-Means as a feature-engineering step	2
2.1	The K-Means objective and Lloyd’s algorithm	2
2.2	From clusters to features	3
2.3	Multinomial logistic regression on the distance features	3
2.4	Why the distance features must be scaled	4
2.5	Choosing k and results	4
3	Part B: Semi-supervised learning with 50 labels	5
3.1	Representative images	5
3.2	Label propagation	5
3.3	How much does one wrong label cost?	6
3.4	Active learning by uncertainty sampling	6
4	Part C: Anomaly detection with Gaussian mixtures	7
4.1	The model	7
4.2	Expectation-Maximisation	7
4.3	Choosing k and the covariance type: BIC and AIC	8
4.4	Reducing the dimension with PCA	9
4.5	The density threshold and its calibration	9
4.6	DBSCAN with a label-free choice of ε	10
4.7	Results	10
5	From experiments to a system	12
6	Limitations	12

1 Data and evaluation protocol

Digits. The scikit-learn digits dataset contains 1,797 grey-scale images of 8×8 pixels, each pixel an integer in $\{0, \dots, 16\}$. We write an image as a vector $\mathbf{x} \in \mathbb{R}^n$ with $n = 64$ and its label as $y \in \{0, \dots, 9\}$. A random 75/25 split (seed 42) gives $m = 1,347$ training and 450 test images, as in the book.

Why several splits. With 450 test images one misclassification moves accuracy by

$$\frac{1}{450} = 0.22 \text{ percentage points (pp)}, \quad (1)$$

so a single split cannot distinguish models that differ by a few tenths of a point. We therefore repeat the key comparisons on five further random splits (seeds $0, \dots, 4$), re-tuning every hyperparameter inside each split on its training set only, and report the mean $\pm$ standard deviation.

Anomaly benchmarks with ground truth. The book’s anomaly-detection examples have no labelled anomalies. We generate the book’s two synthetic datasets (three Gaussian blobs with weights 0.4, 0.4, 0.2, and the two moons) and inject a known contamination $q = 4\%$ of uniformly scattered outliers. Outliers are rejection-sampled so that they are genuinely anomalous: at least 3 Mahalanobis units from every blob, or at least 0.15 from every moon point. With 1,000 inliers this gives

$$n_{\text{out}} = \text{round}\left(1000 \cdot \frac{q}{1-q}\right) = \text{round}\left(\frac{40}{0.96}\right) = 42 \quad (2)$$

outliers per dataset, i.e. a contamination of $42/1042 = 4.03\%$.

Metrics. For a detector that flags a set F of points, with P the set of true outliers,

$$\text{precision} = \frac{|F \cap P|}{|F|}, \quad \text{recall} = \frac{|F \cap P|}{|P|}, \quad F_1 = \frac{2 \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}. \quad (3)$$

ROC-AUC is computed from a continuous anomaly score and does not depend on the threshold.

2 Part A: K-Means as a feature-engineering step

2.1 The K-Means objective and Lloyd’s algorithm

K-Means partitions the training set into k clusters with centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k$ by minimising the *inertia* (within-cluster sum of squares)

$$J(c, \boldsymbol{\mu}) = \sum_{i=1}^m \|\mathbf{x}_i - \boldsymbol{\mu}_{c(i)}\|^2, \quad (4)$$

where $c(i) \in \{1, \dots, k\}$ is the cluster of $\mathbf{x}_i$. Lloyd’s algorithm alternates two steps:

$$\text{assignment: } c(i) \leftarrow \arg \min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2, \quad (5)$$

$$\text{update: } \boldsymbol{\mu}_j \leftarrow \frac{1}{|C_j|} \sum_{i \in C_j} \mathbf{x}_i, \quad C_j = \{i : c(i) = j\}. \quad (6)$$

Proposition 1. The centroid update minimises the inertia of a fixed cluster

For a fixed assignment, the contribution of cluster j to J is $J_j(\boldsymbol{\mu}_j) = \sum_{i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$. Expanding the squared norm,

$$J_j(\boldsymbol{\mu}_j) = \sum_{i \in C_j} (\mathbf{x}_i^\top \mathbf{x}_i - 2 \mathbf{x}_i^\top \boldsymbol{\mu}_j + \boldsymbol{\mu}_j^\top \boldsymbol{\mu}_j).$$

Differentiating with respect to $\boldsymbol{\mu}_j$ term by term,

$$\nabla_{\boldsymbol{\mu}_j} J_j = \sum_{i \in C_j} (-2 \mathbf{x}_i + 2 \boldsymbol{\mu}_j) = -2 \sum_{i \in C_j} \mathbf{x}_i + 2 |C_j| \boldsymbol{\mu}_j.$$

Setting the gradient to zero gives $\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{i \in C_j} \mathbf{x}_i$, which is (6). The Hessian is $\nabla_{\boldsymbol{\mu}_j}^2 J_j = 2 |C_j| I \succ 0$, so J_j is strictly convex and this stationary point is the unique minimiser. $\square$

Proposition 2. Lloyd’s algorithm terminates

The assignment step (5) chooses, for each i separately, the centroid that minimises its own term of (4), so it cannot increase J . By Proposition 1 the update step minimises each J_j for the current assignment, so it cannot increase J either. Hence J is non-increasing along the iterations. There are at most k^m assignments, and each is visited at most once while J strictly decreases (ties broken consistently). Therefore the algorithm reaches a fixed point after finitely many iterations. The fixed point is a local minimum in general, which is why scikit-learn restarts from `n_init=10` random initialisations and keeps the solution with the lowest J . $\square$

2.2 From clusters to features

After fitting, every image is mapped to its vector of distances to the k centroids,

$$\phi(\mathbf{x}) = (\|\mathbf{x} - \boldsymbol{\mu}_1\|, \|\mathbf{x} - \boldsymbol{\mu}_2\|, \dots, \|\mathbf{x} - \boldsymbol{\mu}_k\|)^\top \in \mathbb{R}^k. \quad (7)$$

This is a non-linear map: a linear model in $\phi(\mathbf{x})$ can carve out regions around each centroid, which a linear model on raw pixels cannot. The features are then standardised component-wise,

$$z_\ell = \frac{\phi_\ell(\mathbf{x}) - \bar{\phi}_\ell}{s_\ell}, \quad \ell = 1, \dots, k, \quad (8)$$

with $\bar{\phi}_\ell$ and s_ℓ the training mean and standard deviation (Section 2.4 explains why).

2.3 Multinomial logistic regression on the distance features

For classes $c = 0, \dots, 9$ with parameter vectors $\boldsymbol{\theta}_c$ (bias absorbed into $\mathbf{z}$), the model is the softmax

$$p_c(\mathbf{z}) = \frac{\exp(\boldsymbol{\theta}_c^\top \mathbf{z})}{\sum_{c'} \exp(\boldsymbol{\theta}_{c'}^\top \mathbf{z})}. \quad (9)$$

With one-hot targets y_{ic} , the regularised cross-entropy minimised by `lbfgs` is

$$L(\boldsymbol{\theta}) = - \sum_{i=1}^m \sum_c y_{ic} \log p_c(\mathbf{z}_i) + \frac{1}{2C} \sum_c \|\boldsymbol{\theta}_c\|^2. \quad (10)$$

Proposition 3. Gradient of the cross-entropy

Write $a_{ic} = \theta_c^\top \mathbf{z}_i$ and $\log p_c(\mathbf{z}_i) = a_{ic} - \log \sum_{c'} e^{a_{ic'}}$. For a fixed class d ,

$$\frac{\partial \log p_c(\mathbf{z}_i)}{\partial a_{id}} = \delta_{cd} - \frac{e^{a_{id}}}{\sum_{c'} e^{a_{ic'}}} = \delta_{cd} - p_d(\mathbf{z}_i).$$

Using $\sum_c y_{ic} = 1$,

$$\frac{\partial}{\partial a_{id}} \left(- \sum_c y_{ic} \log p_c(\mathbf{z}_i) \right) = - \sum_c y_{ic} (\delta_{cd} - p_d(\mathbf{z}_i)) = p_d(\mathbf{z}_i) - y_{id}.$$

Since $\partial a_{id} / \partial \theta_d = \mathbf{z}_i$, the chain rule gives

$$\nabla_{\theta_d} L = \sum_{i=1}^m (p_d(\mathbf{z}_i) - y_{id}) \mathbf{z}_i + \frac{1}{C} \theta_d.$$

□

2.4 Why the distance features must be scaled

The gradient in Proposition 3 is a weighted sum of the feature vectors $\mathbf{z}_i$, so the curvature of L is governed by the Gram matrix $Z^\top Z$ of the features (with a column of ones for the bias). Raw distances are all positive and large compared with their spread: for $k = 50$ the median distance is 45, most values lie between 29 and 56, and the average per-feature standard deviation is only 7.7. The common offset creates one direction of very large curvature next to many directions of small curvature. The condition number of $Z^\top Z$ is 2.2×10^8 on raw distances and 4.7×10^3 after standardisation, a reduction of more than four orders of magnitude. A quasi-Newton method converges much more slowly on an ill-conditioned problem. On our split with $k = 50$, `lbfgs` converged in 44 iterations on scaled features versus 1,268 on raw distances, and during the full grid search some raw-distance fits stopped at the 10,000-iteration limit. With an identical grid ($k = 10, 15, \dots, 95$), the scaled variant reached a cross-validated accuracy of 95.9% against 95.6%, with the same test accuracy (97.6%), and fitted the grid 6.3 times faster (7 s versus 46 s).

2.5 Choosing k and results

Because K-Means is only a preprocessing step, the right k is the one that maximises the *classifier's* cross-validated accuracy, not the one that minimises inertia or maximises the silhouette score. A 3-fold `GridSearchCV` over $k = 2, \dots, 99$ on the training set selects $k = 81$ (CV accuracy 96.0%). Figure 1 shows the unsupervised criteria preferring far smaller k .

Unsupervised criteria (grey) favour small k ; the classifier prefers many more clusters

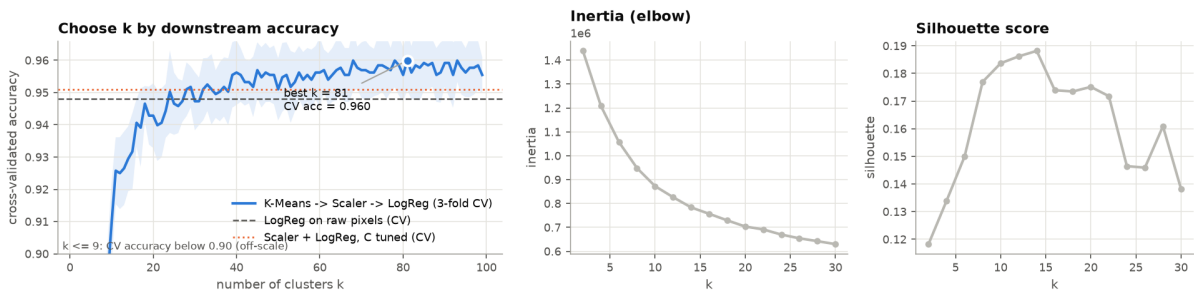


Figure 1: Left: CV accuracy of the K-Means → Logistic Regression pipeline versus k (± 1 std band); dashed line: Logistic Regression on raw pixels. Middle and right: inertia and silhouette score.

A fair comparison. The book compares against Logistic Regression on raw pixels with default settings, while the pipeline also benefits from scaling and from a 98-value search over k . We therefore add a strong baseline: StandardScaler followed by Logistic Regression with the regularisation strength C tuned by the same 3-fold cross-validation over $C \in \{0.01, 0.03, \dots, 100\}$. On the five extra splits, every hyperparameter (k on the coarse grid 10, 20, $\dots$, 90, and C) is re-tuned on each split’s training set.

Model	Split 42	Mean $\pm$ std over 5 splits
Logistic Regression, raw pixels (book)	97.3%	95.6% $\pm$ 1.0%
StandardScaler $\rightarrow$ LogReg (C tuned)	97.1%	96.3% $\pm$ 1.2%
K-Means $\rightarrow$ StandardScaler $\rightarrow$ LogReg (k tuned)	97.6%	96.9% $\pm$ 0.9%

Table 1: Part A test accuracy. On split 42 the difference between the book baseline and the pipeline is 12 $\rightarrow$ 11 errors. Over five splits the pipeline beats the book baseline on 5/5 splits (+1.29 pp) and the strong baseline on 4/5 (+0.62 pp).

Summary: Part A

- Distances to K-Means centroids are a useful non-linear feature map for a linear classifier. The gain is +0.6 pp over a tuned, scaled baseline (better on 4/5 splits) and +1.3 pp over the book’s baseline. It is real but modest, and close to the resolution of a 450-image test set.
- k is a classifier hyperparameter and is chosen by cross-validated accuracy ($k = 81$), not by inertia.
- Standardising the distances fixes the conditioning of the optimisation: 44 instead of 1,268 iterations, a $6\times$ faster grid search, and equal accuracy.

3 Part B: Semi-supervised learning with 50 labels

3.1 Representative images

Suppose only $k = 50$ images can be labelled (3.7% of the training set). Cluster the training set into k clusters and, for each centroid, pick the image closest to it:

$$r_j = \arg \min_{i \in \{1, \dots, m\}} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|, \quad j = 1, \dots, k. \quad (11)$$

These *representative* images cover the modes of the data, whereas 50 random images over-sample common writing styles and miss rare ones. A human (or, in reproducible runs, the ground truth acting as an oracle) labels $y_{r_1}, \dots, y_{r_k}$.

3.2 Label propagation

Full propagation gives every image the label of its cluster’s representative,

$$\hat{y}_i = y_{r_{c(i)}}. \quad (12)$$

Images near a cluster boundary are often a different digit, so we keep only the images closest to their centroid. With $d_i = \|\mathbf{x}_i - \boldsymbol{\mu}_{c(i)}\|$ and $Q_j(p)$ the p -th percentile of $\{d_i : i \in C_j\}$, the *partially propagated* training set is

$$S_p = \bigcup_{j=1}^k \{i \in C_j : d_i \leq Q_j(p)\} \cup \{r_1, \dots, r_k\}. \quad (13)$$

Smaller p means fewer but more reliable pseudo-labels. On our split, $p = 20$ keeps 288 images whose propagated labels are 98.3% correct, against 93.5% for full propagation (Figure 2).

3.3 How much does one wrong label cost?

Let $n_j = |C_j|$. With partial propagation, cluster j contributes about $p n_j/100$ images to S_p , so

$$|S_p| \approx \frac{p}{100} \sum_{j=1}^k n_j = \frac{p m}{100}. \quad (14)$$

If the representative label of cluster j is wrong, every one of its propagated images is wrong, which is a share

$$\frac{p n_j/100}{p m/100} = \frac{n_j}{m} \approx \frac{1}{k} = \frac{1}{50} = 2\% \quad (15)$$

of the training set. These mislabelled images all lie in one region of input space, so the model learns a wrong decision there rather than averaging out the noise. Empirically, one random wrong representative label lowers test accuracy by 2.4 pp on average (92.4% $\rightarrow$ 90.1%, worst case 88.4%), two by 5.3 pp, and five by 12.5 pp. In ordinary supervised learning one wrong label out of 1,347 would be invisible. This sensitivity is the reason the deployment pipeline has a champion-versus-challenger gate (Section 5).

3.4 Active learning by uncertainty sampling

To spend further labels well, the model asks about the unlabelled images it is least sure of:

$$i^* = \arg \min_{i \notin S} \max_c p_c(\mathbf{x}_i). \quad (16)$$

Adding 10 such labels per round for 5 rounds (50 $\rightarrow$ 100 human labels) raises test accuracy from 92.4% to 93.8%.

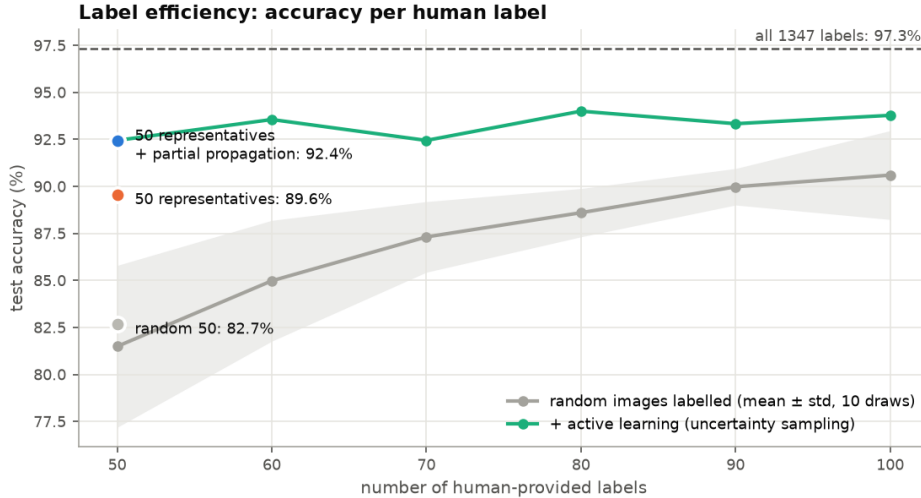


Figure 2: Test accuracy versus number of human labels. Grey: random images (mean $\pm$ std over 10 draws). Dots at 50 labels: random, representatives, representatives with partial propagation. Aqua: active learning.

Training labels	Images	Label accuracy	Test accuracy
50 random images	50	100%	82.7%
50 representatives	50	100%	89.6%
+ full propagation	1,347	93.5%	90.9%
+ partial propagation ($p = 20$)	288	98.3%	92.4%
All true labels (upper bound)	1,347	100%	97.3%

Table 2: Part B on split 42. Over five further splits: random 50 labels $83.4\% \pm 2.5\%$ versus partial propagation $92.4\% \pm 1.0\%$, with propagated-label accuracy 98.9%.

Summary: Part B

- Choosing *which* 50 images to label matters: representatives plus partial propagation reach 92.4% against 82.7% for random labels, using 3.7% of the labels.
- Partial propagation trades coverage for label quality (98.3% correct pseudo-labels).
- Each representative label stands for about $1/k$ of the training data, so labelling mistakes are amplified. Label quality must be checked before deployment.

4 Part C: Anomaly detection with Gaussian mixtures

4.1 The model

A Gaussian mixture model (GMM) assumes each instance is generated by first drawing a cluster $z \sim \text{Categorical}(\phi_1, \dots, \phi_k)$ and then $\mathbf{x} \mid z = j \sim \mathcal{N}(\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$. Marginalising over z gives the density

$$p(\mathbf{x}) = \sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j), \quad \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{\exp(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}))}{(2\pi)^{n/2} |\boldsymbol{\Sigma}|^{1/2}}, \quad (17)$$

with $\phi_j \geq 0$ and $\sum_j \phi_j = 1$. The log-likelihood of the training set is

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^m \log \sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j). \quad (18)$$

The logarithm of a sum has no closed-form maximiser, which motivates the Expectation-Maximisation (EM) algorithm.

4.2 Expectation-Maximisation

E-step. With the current parameters, Bayes' theorem gives the *responsibility* of cluster j for instance i :

$$\gamma_{ij} = P(z_i = j \mid \mathbf{x}_i) = \frac{\phi_j \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\sum_{l=1}^k \phi_l \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_l, \boldsymbol{\Sigma}_l)}. \quad (19)$$

M-step. Holding the γ_{ij} fixed, maximise the expected complete-data log-likelihood

$$Q(\boldsymbol{\theta}) = \sum_{i=1}^m \sum_{j=1}^k \gamma_{ij} \left[\log \phi_j - \frac{n}{2} \log 2\pi - \frac{1}{2} \log |\boldsymbol{\Sigma}_j| - \frac{1}{2} (\mathbf{x}_i - \boldsymbol{\mu}_j)^\top \boldsymbol{\Sigma}_j^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_j) \right]. \quad (20)$$

Write $N_j = \sum_i \gamma_{ij}$ for the effective number of instances in cluster j .

Proposition 4. M-step update of the weights

Only the term $\sum_{i,j} \gamma_{ij} \log \phi_j = \sum_j N_j \log \phi_j$ depends on ϕ . We maximise it subject to $\sum_j \phi_j = 1$ with a Lagrange multiplier λ :

$$\mathcal{L}(\phi, \lambda) = \sum_{j=1}^k N_j \log \phi_j + \lambda \left(1 - \sum_{j=1}^k \phi_j\right).$$

Setting $\partial \mathcal{L} / \partial \phi_j = N_j / \phi_j - \lambda = 0$ gives $\phi_j = N_j / \lambda$. Summing over j and using the constraint, $1 = \sum_j N_j / \lambda = m / \lambda$, since $\sum_j \gamma_{ij} = 1$ for every i . Hence $\lambda = m$ and

$$\phi_j = \frac{N_j}{m}.$$

□

Proposition 5. M-step update of the means

The terms of (20) involving μ_j are $-\frac{1}{2} \sum_i \gamma_{ij} (\mathbf{x}_i - \mu_j)^\top \Sigma_j^{-1} (\mathbf{x}_i - \mu_j)$. Since $\nabla_{\mathbf{a}} \mathbf{a}^\top A \mathbf{a} = 2A\mathbf{a}$ for symmetric A , and $\partial(\mathbf{x}_i - \mu_j) / \partial \mu_j = -I$,

$$\nabla_{\mu_j} Q = \sum_{i=1}^m \gamma_{ij} \Sigma_j^{-1} (\mathbf{x}_i - \mu_j) = \Sigma_j^{-1} \left(\sum_i \gamma_{ij} \mathbf{x}_i - N_j \mu_j \right).$$

Σ_j^{-1} is invertible, so setting the gradient to zero gives

$$\mu_j = \frac{1}{N_j} \sum_{i=1}^m \gamma_{ij} \mathbf{x}_i,$$

a responsibility-weighted average. With hard responsibilities ($\gamma_{ij} \in \{0, 1\}$) this is exactly the K-Means update (6). This is the sense in which EM generalises K-Means. □

Proposition 6. M-step update of the covariances

Let $\Lambda_j = \Sigma_j^{-1}$ and use $\log |\Sigma_j| = -\log |\Lambda_j|$ and $(\mathbf{x} - \mu)^\top \Lambda (\mathbf{x} - \mu) = \text{tr}(\Lambda (\mathbf{x} - \mu)(\mathbf{x} - \mu)^\top)$. The Λ_j -dependent part of (20) is

$$Q_j(\Lambda_j) = \frac{1}{2} N_j \log |\Lambda_j| - \frac{1}{2} \text{tr} \left(\Lambda_j \sum_i \gamma_{ij} (\mathbf{x}_i - \mu_j)(\mathbf{x}_i - \mu_j)^\top \right).$$

Using the matrix identities $\nabla_{\Lambda} \log |\Lambda| = \Lambda^{-1}$ and $\nabla_{\Lambda} \text{tr}(\Lambda B) = B^\top$,

$$\nabla_{\Lambda_j} Q_j = \frac{1}{2} N_j \Lambda_j^{-1} - \frac{1}{2} \sum_i \gamma_{ij} (\mathbf{x}_i - \mu_j)(\mathbf{x}_i - \mu_j)^\top = 0 \implies \Sigma_j = \frac{1}{N_j} \sum_{i=1}^m \gamma_{ij} (\mathbf{x}_i - \mu_j)(\mathbf{x}_i - \mu_j)^\top.$$

scikit-learn adds $\text{reg_covar} \cdot I = 10^{-6} I$ to keep Σ_j positive definite. □

Each EM iteration cannot decrease $\ell(\theta)$, but, like K-Means, EM can converge to a poor local optimum. It is therefore restarted from several initialisations (`n_init`) and the best run is kept.

4.3 Choosing k and the covariance type: BIC and AIC

Inertia and the silhouette score assume spherical, equally sized clusters, so for GMMs we use information criteria. With $\hat{L}$ the maximised likelihood, p the number of free parameters and m

the number of instances,

$$\text{BIC} = p \log m - 2 \log \hat{L}, \quad (21)$$

$$\text{AIC} = 2p - 2 \log \hat{L}. \quad (22)$$

Both reward fit and penalise complexity; BIC's penalty grows with m and selects simpler models.

Parameter counts for k components in n dimensions

There are $k - 1$ free weights (they sum to one) and kn mean coordinates. A symmetric $n \times n$ covariance has $n(n + 1)/2$ free entries. Therefore

$$\begin{aligned} p_{\text{full}} &= (k - 1) + kn + k \frac{n(n+1)}{2}, & p_{\text{tied}} &= (k - 1) + kn + \frac{n(n+1)}{2}, \\ p_{\text{diag}} &= (k - 1) + kn + kn, & p_{\text{spherical}} &= (k - 1) + kn + k. \end{aligned}$$

For the digits detector selected below ($k = 6$, full covariance, $n = 28$ after PCA):

$$p = 5 + 6 \cdot 28 + 6 \cdot \frac{28 \cdot 29}{2} = 5 + 168 + 2436 = 2609,$$

and the BIC penalty is $p \log m = 2609 \cdot \log(1347) = 2609 \cdot 7.206 \approx 18,800$ nats.

4.4 Reducing the dimension with PCA

In 64 dimensions a single full covariance already has $64 \cdot 65/2 = 2080$ parameters, against 1,347 training images. Following the book's Chapter 8, we first project onto the d leading principal components, choosing the smallest d that keeps 95% of the variance:

$$d = \min \left\{ d' : \frac{\sum_{i=1}^{d'} \lambda_i}{\sum_{i=1}^n \lambda_i} \geq 0.95 \right\}, \quad (23)$$

where $\lambda_1 \geq \lambda_2 \geq \dots$ are the eigenvalues of the training covariance matrix. This gives $d = 28$. BIC over $k = 1, \dots, 15$ and the four covariance types then selects $k = 6$ with full covariances.

4.5 The density threshold and its calibration

The anomaly score is the log-density $s(\mathbf{x}) = \log p(\mathbf{x})$. Given the expected contamination $q = 4\%$, the book sets the threshold τ to the q -quantile of the training scores and flags $\mathbf{x}$ when $s(\mathbf{x}) < \tau$.

This threshold is optimistic. EM chooses θ to maximise the sum of the *training* scores, so training images have systematically higher density than unseen images from the same distribution. A threshold taken from training scores therefore flags more than q of new normal data. We measured 8.9% instead of 4% on the clean test digits. The fix is to fit the GMM on 75% of the training images and take τ from the held-out 25%.

Proposition 7. A held-out quantile controls the false-alarm rate

Let $s_1, \dots, s_N$ be the scores of N held-out normal images and s_{N+1} the score of a new normal image. Assume the fitted model is independent of these $N + 1$ images and the scores are exchangeable with no ties. Then the rank R of s_{N+1} among all $N + 1$ scores is uniformly distributed on $\{1, \dots, N + 1\}$. Let $\tau = s_{(\ell)}$ be the ℓ -th smallest held-out score. The new image is flagged exactly when it ranks below $s_{(\ell)}$, so

$$P(s_{N+1} < s_{(\ell)}) = P(R \leq \ell) = \frac{\ell}{N + 1}.$$

Choosing $\ell \approx qN$ makes the expected false-alarm rate approximately q . Here $N = 337$ and the empirical 4th percentile lies between the 14th and 15th smallest held-out scores

($\ell \approx 14.4$), so the expected rate is $14.4/338 \approx 4.3\%$. The observed 2.9% (13 of 450 clean test images) is about 1.5 binomial standard deviations below this, where one standard deviation is $\sqrt{0.043 \cdot 0.957/450} \approx 0.96$ pp. Training scores do not satisfy the exchangeability assumption, because the model was fitted to them. This is exactly why the training quantile fails. $\square$

4.6 DBSCAN with a label-free choice of ε

DBSCAN marks an instance as a *core* instance if at least `min_samples` = 5 instances lie within distance ε . Clusters are chains of neighbouring core instances, and instances that are neither core nor near one are *noise*, which we treat as anomalies. The result is very sensitive to ε : on the moons, $\varepsilon = 0.05$ gives 7 clusters and 11.4% noise, while $\varepsilon = 0.2$ gives the 2 moons. To compare DBSCAN fairly with the GMM, we give it the same prior and *no labels*: we choose ε on a geometric grid so that the noise fraction is closest to q .

Why precision equals recall in the GMM rows. With the threshold at the 4th percentile of the $M = 1042$ scores, the GMM flags $|F| = 42$ points, which is exactly the number of injected outliers $|P| = 42$. When $|F| = |P|$,

$$\text{precision} = \frac{|F \cap P|}{|F|} = \frac{|F \cap P|}{|P|} = \text{recall} = F_1. \quad (24)$$

DBSCAN’s noise fraction only approximately equals q , so its precision and recall differ slightly.

4.7 Results

F_1 (mean $\pm$ std over 5 random datasets)	Blobs (elliptical)	Moons (curved)
GMM, k and covariance type by BIC	0.914 $\pm$ 0.013	0.971 $\pm$ 0.011
DBSCAN, label-free ε	0.884 $\pm$ 0.024	1.000 $\pm$ 0.000
Local Outlier Factor	0.857 $\pm$ 0.048	1.000 $\pm$ 0.000
Isolation Forest	0.819 $\pm$ 0.071	0.810 $\pm$ 0.053
Elliptic Envelope (Fast-MCD)	0.667 $\pm$ 0.081	0.652 $\pm$ 0.055

Table 3: Anomaly benchmark with 4% injected outliers. The last three detectors are the book’s other methods, included for reference. Fast-MCD assumes a single Gaussian and fails on multi-cluster data.

On clean test digits, images that the detector flags are misclassified by the supervised model 30.8% of the time (4 of 13), against 1.6% (7 of 437) for the rest. The detector is therefore a useful signal of when not to trust a prediction. This matters because the classifier itself cannot abstain: softmax always outputs one of the ten digits. On pure noise its average confidence is still 65%, and the detector flags all of these images.

Summary: Part C

- A GMM gives every input a calibrated “have I seen data like this?” score. EM generalises K-Means with soft assignments, and BIC/AIC choose the number and shape of the components.
- On elliptical clusters the GMM is the best detector ($F_1 = 0.91$). On curved clusters DBSCAN is ($F_1 = 1.00$). The choice depends on which assumptions match the data.
- The threshold must be set on held-out densities. A training-set quantile is optimistic and flagged 8.9% of clean digits instead of 4%.

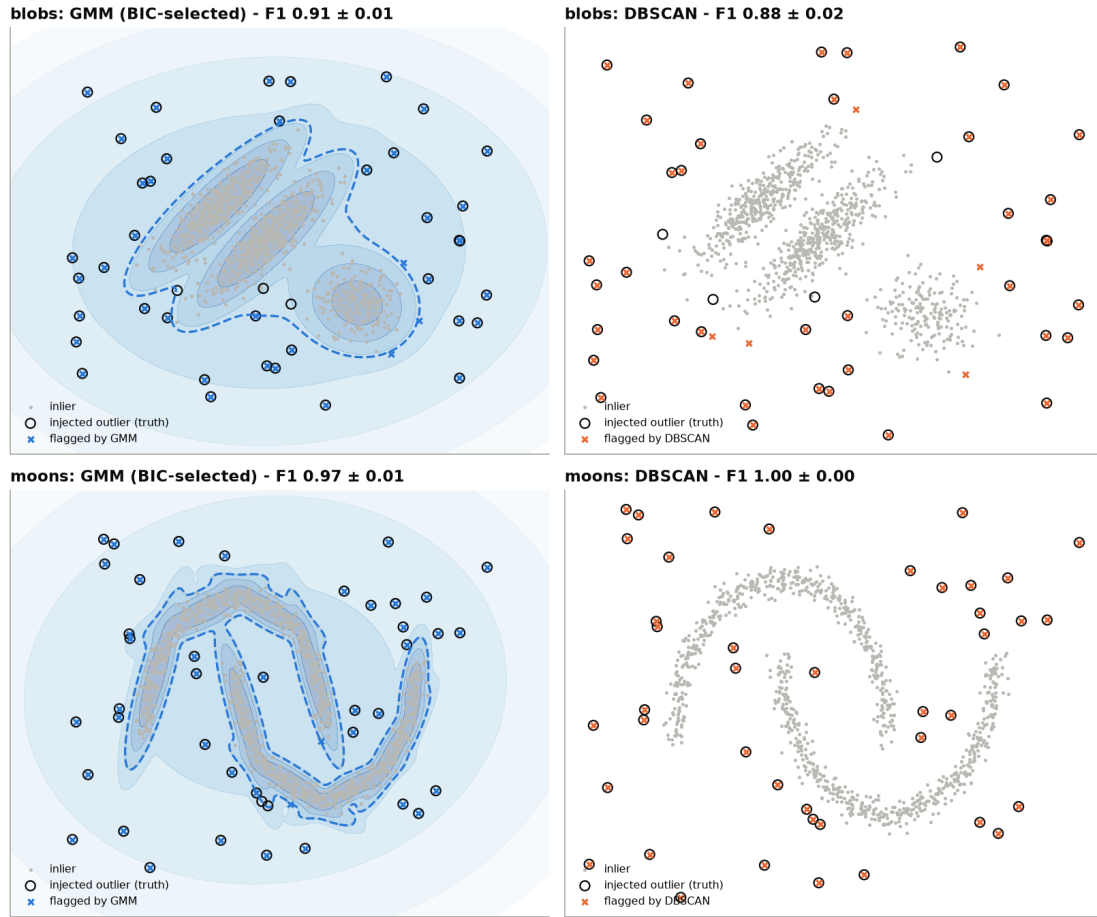


Figure 3: GMM (left, with density contours and the threshold as a dashed line) and DBSCAN (right) on one blob dataset and one moon dataset. Circles mark the true outliers and crosses the flagged points.

- Blind spots are reported, not hidden: dimmed strokes (6% caught) and mild Gaussian noise (34%).

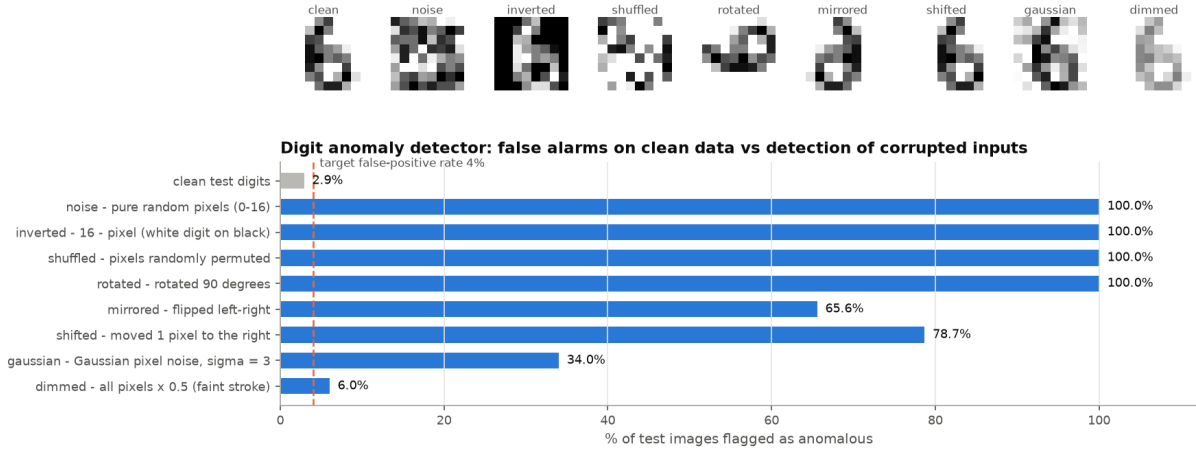


Figure 4: Digit detector. False alarms on clean test digits (2.9%) and detection rate for eight kinds of corrupted input. Structural corruptions are caught every time; dimmed digits and mild noise mostly are not.

5 From experiments to a system

The three models are trained, evaluated and served by one configuration file and one command (`docker compose up -build`). The orchestration has three stages.

1. **Train** (Prefect). Parts A, B and C run as tasks. Parameters, metrics, tables and figures are logged to MLflow, and the three models are registered in the MLflow Model Registry (skops serialisation).
2. **Evaluate**. The registered candidates are re-scored on the test set and must pass every gate:

$$\text{acc}_{\text{sup}} \geq 0.95, \quad \text{acc}_{\text{semi}} \geq 0.85, \quad \text{FPR}_{\text{clean}} \leq 8\%, \quad \text{DR}_{\text{noise, inverted, shuffled}} \geq 95\%, \quad (25)$$

and must not be worse than the current champion by more than a tolerance $\delta = 0.5$ pp:

$$\text{acc}_{\text{candidate}} \geq \text{acc}_{\text{champion}} - \delta. \quad (26)$$

3. **Deploy**. The `@champion` alias moves to the new versions and the FastAPI service hot-reloads them.

Every prediction is logged to PostgreSQL. A monitoring flow raises an alert when the live anomaly rate exceeds $3q = 12\%$, because by Proposition 7 normal traffic should be flagged at about q . The web interface lets a person label the 50 representative images and feed them back into Part B. In a test with one deliberate labelling mistake, the retrained semi-supervised model fell from 92.4% to 88.9%. The champion-versus-challenger gate rejected it, so the better model stayed in production. This is the amplification effect derived in Part B, observed in the running system.

6 Limitations

- The digits dataset is small (1,797 images of 8×8). The methods transfer to other domains, but the absolute numbers do not.
- Injected outliers and synthetic corruptions are cleaner than real incidents. Precision and recall should be re-measured on labelled real anomalies before relying on the detector.
- The density detector misses dimmed and mildly noisy digits. A complementary score, such as the PCA reconstruction error, could cover these cases.

-
- Active learning is simulated with an oracle. Routing its queries to the labelling page would test it with a real annotator.
 - The same 450 test images are used for the deployment gates and for the reported results. On a larger dataset a separate validation split should drive the gates.

Reference. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 2nd ed., O'Reilly Media, 2019. Chapter 8 (Dimensionality Reduction) and Chapter 9 (Unsupervised Learning Techniques).